Laboratorium 09 – Obsługa danych JSON

Wczytywanie i wyświetlanie danych 📄

Aplikacje rzadko działają tylko na „twardo wpisanych” tekstach. Najczęściej muszą korzystać z danych zapisanych w plikach lub pobranych z API. W Flutterze dane w formacie **JSON** można wczytać z pliku `assets` albo z internetu i zamienić je na obiekty w Dart.

---

1. Przygotowanie pliku JSON

Utwórz w projekcie folder:

```
assets/data/
```

Dodaj w nim plik `users.json`

```
[
  {"name": "Anna", "age": 22, "city": "Kraków"},
  {"name": "Michał", "age": 28, "city": "Warszawa"},
  {"name": "Darek", "age": 19, "city": "Gdańsk"}
]
```

W pliku `pubspec.yaml` dopisz assety:

```
flutter:
  assets:
    - assets/data/users.json
```

👉 Dzięki temu Flutter będzie wiedział, że plik JSON jest częścią aplikacji.

---

2. Wczytanie pliku JSON

W kodzie potrzebujemy `rootBundle` i `dart:convet`:

```
import 'dart:convert';
import 'package:flutter/services.dart' show rootBundle;
```

Wczytanie danych:

```
Future<List<dynamic>> loadUsers() async {
  final String jsonString = await rootBundle.loadString('assets/data/
users.json');
  return json.decode(jsonString);
}
```

👉 Funkcja zwraca listę map (`List<dynamic>`), gdzie każdy element to obiekt użytkownika.

---

3. Wyświetlenie danych w liście

Możemy użyć `FutureBuilder`, który poczeka na dane i pokaże listę:

```
FutureBuilder(
  future: loadUsers(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const CircularProgressIndicator();
    }
    if (!snapshot.hasData) {
      return const Text('Brak danych');
    }
    final users = snapshot.data as List;
    return ListView.builder(
      itemCount: users.length,
      itemBuilder: (context, i) {
        final user = users[i];
        return ListTile(
          title: Text(user['name']),
          subtitle: Text('${user['age']} lat, ${user['city']}'),
        );
      },
    );
  },
)
```

👉 **Wyjaśnienie:**

- `FutureBuilder` = widget, który reaguje na wynik przyszłej operacji (`Future`).
- Jeśli dane się wczytują → pokazuje `spinner`.
- Jeśli są gotowe → buduje listę użytkowników.

4. Modelowanie danych

Zamiast pracować na `Map<String, dynamic>`, lepiej zrobić klasę modelu.

```
class User {
  final String name;
  final int age;
  final String city;

  User({required this.name, required this.age, required this.city});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      name: json['name'],
      age: json['age'],
      city: json['city'],
    );
  }
}
```

Użycie:

```
final users = (snapshot.data as List)
    .map((json) => User.fromJson(json))
    .toList();
```

👉 Teraz zamiast `user['name']` możemy pisać `user.name`.

Zadania praktyczne 💻

1. **Lista użytkowników**

- Przygotuj plik `users.json` (możesz użyć przykładowego z sekcji 1).
- Wczytaj go w aplikacji i wyświetl listę imion w `ListView`.

2. **Szczegóły użytkownika**

- Po kliknięciu w element listy otwórz nowy ekran.
- Wyświetl tam imię, wiek i miasto użytkownika.

3. **Rozbudowana karta**

- Zamiast zwykłego `ListTile` użyj `Card`.
- W środku pokaż imię pogrubione, wiek i miasto w mniejszym tekście, a po prawej ikonę profilu (`Icons.person`).

Przydatne linki 🔗

- **JSON i Flutter Cookbook**: https://docs.flutter.dev/cookbook/networking/fetch-data
- **dart:convert**: https://api.dart.dev/stable/dart-convert/dart-convert-library.html
- **FutureBuilder**: https://api.flutter.dev/flutter/widgets/FutureBuilder-class.html